Automated Cloud Deployment Using Terraform, Ansible, Docker and GitHub Actions on Microsoft Azure

Name: Aditya Mahendra Rawat

Module Title: Network Systems & Administration (B9IS121)

Lecturer: Kingsley Ibomo

Assessment: CA-1 — Automated Cloud Deployment

GitHub Repository: devops-ca-project/README.md at main · Addyrawat/devops-ca-project

Project Video Link: https://drive.google.com/file/d/1McEQGXptjYE45qPUJFeCFYUdG-Vyc_NA/view?usp=drive_link

Content

Introduction ……..…… 4

Development Environment — windows Subsystem for Linux(WSL) to ensure compatibility …….…… 4

Project Architecture …… 5

Azure Infrastructure Design …… 5

Resource Group …… 5

Virtual Network (VNet) …… 5

Subnet …… 5

Network Security Group (NSG) …… 6

Public IP and Network Interface …… 6

Linux Virtual Machine …… 6

Infrastructure Provisioning Using Terraform …… 6

Terraform Workflow …… 6

Configuration Management Using Ansible …… 8

Containerization Using Docker …… 9

Continuous Integration and Continuous Deployment (CI/CD) …… 10

Security and Cost Management …… 10

Reflection and Learning Outcomes …… 11

Conclusion …… 11

References …… 12

Annexure …… 12

Annexure A – Terraform Configuration Snippet (main.tf) …… 12

Annexure B – Ansible Playbook Snippet (deploy.yml) …… 16

Annexure C – Project Architecture …… 17

Table Of Figures

Fig 1.1 Terraform Init Command 7

Fig 1.2 Terraform plan Command..... 7

Fig 1.3 Terraform Apply Command 7

Fig 1.4 Terraform Apply Command and Public IP obtained 7

Fig 2.1 Ansible Ping Command ……..... 8

Fig 2.2 Ansible Playbook Command ……..... 8

Fig 3.1 Docker Build Command 9

Fig 3.2 Docker Run Command ……..……..... 9

Fig 3.3 Docker Ps Command ……. 9

Fig 4.1 GitHub Repository Secrets ……. 10

Fig 13.3 Project Architecture Diagram ……... 17

1. Introduction

Cloud computing has changed the way that modern IT systems are set up, run, and kept up.

Setting up traditional infrastructure required manual configuration, which took a lot of time, was easy to make mistakes, and was hard to repeat. Infrastructure as Code(IaC), configuration management, containerisation, and CI/CD pipelines are all DevOps practices that help with these problems by making automation, scalability, and continuous delivery possible.

This project shows how to design and build an automated cloud deployment system with Microsoft Azure. Terraform sets up the infrastructure, Ansible sets up the server, Docker runs the application, and GitHub Action automates, scalability, and continuous delivery possible.

The main goal was to create a cloud deployment pipeline that is safe, repeatable, and cost-effective, and the mimics real DevOps processes used in business settings

---

2. Development Environment — windows Subsystem for Linux(WSL) to ensure compatibility

To ensure compatibility with Linux-Based DevOps tools, the development environment was configured using windows subsystem for Linux (WSL) running Ubuntu. WSL allows Linux application and commands to run directly on Windows without the overhead of a full virtual machine.

Using WSL provided:

Native Linux shell environment

Support for SSH, Terraform, Ansible, Docker CLI

Compatibility with cloud tooling

Simplified package management

A pair of SSH keys was made for safe authentication:

```
ssh-keygen -t rsa -b 4096
```

The pubic key was uploaded to the Azure VM, enabling passwords-less login using SSH

3. Project Architecture

The system is built on a layered cloud architecture that includes infrastructure, configuration, application, and automation layers (Azure)

Infrastructure Layer (Azure)
Provisioned using Terraform and includes networking and compute resources.

Configuration Layer
Ansible prepares the VM by installing required software such as Docker and Git.

Application Layer
The web application runs inside a Docker container using Nginx

Automation Layer
GitHub Actions implements CI/CD for automatic deployment when changes are pushed.

A similar layered architecture is described in the reference report (see Project Architecture section, pages 4–5), but this implementation was independently designed and executed.

---

4. Azure Infrastructure Design

All cloud resources were created within a single Azure Resource Group to simplify management and lifecycle control.

4.1 Resource Group

A Resource Group is a logical container for Azure resources that are related to each other.It lets you monitor and delete them all in one place

4.2 Virtual Network (VNet)

A Virtual Network was created to enable isolated communication between cloud components, a virtual network was developed. Secure internal networking is ensured by private IP addressing.

4.3 Subnet

A subnet was made inside the VNet to host the virtual machine. This made it easier to control and separate traffic.

4.4 Network Security Group (NSG)

The NSG functions as a firewall. Only essential inbound ports were allowed:

SSH (PORT 22) — remote management

HTTP (Port 80) — web access

Restricting access reduces the attack surface.

4.5 Public IP and Network Interface

To enable external access, a static public IP address was assigned. The virtual machine is linked to the public IP and subnet via the network interface.

4.6 Linux Virtual Machine

An Ubuntu Linux VM was deployed to host the application environment. SSH key authentication was enabled for security.

---

5. Infrastructure Provisioning Using Terraform

Terraform was used to write code that declaratively defined and deployed Azure resources.

5.1 Terraform Workflow

Key commands used:

```
terraform init
terraform plan
terraform apply
terraform destroy
```

init — downloads provider plugins

plan — previews infrastructure changes

apply — creates resources

destroy — removes infrastructure

Terraform is idempotent nature ensures that if resources already exist in the desired state, they are reused rather than recreated.

After deployment, Terraform outputs the public IP address of the VM, which is required for remote access and configuration

```
aditya@LAPTOP-J5IGE3UJ:~/devops-ca-project/terraform$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/azurerm from the dependency lock file
- Using previously-installed hashicorp/azurerm v3.117.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Fig 1.1 Terraform Init Command

```
aditya@LAPTOP-J5IGE3UJ:~/devops-ca-project/terraform$ terraform plan
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg]
azurerm_public_ip.public_ip: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/provid
soft.Network/publicIPAddresses/devops-ca-public-ip]
azurerm_virtual_network.vnet: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/provi
osoft.Network/virtualNetworks/devops-ca-vnet]
azurerm_network_security_group.nsg: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg
s/Microsoft.Network/networkSecurityGroups/devops-ca-nsg]
azurerm_subnet.subnet: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers/Mi
etwork/virtualNetworks/devops-ca-vnet/subnets/devops-ca-subnet]
azurerm_network_security_rule.http: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg
s/Microsoft.Network/networkSecurityGroups/devops-ca-nsg/securityRules/Allow-HTTP]
azurerm_network_security_rule.ssh: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/
/Microsoft.Network/networkSecurityGroups/devops-ca-nsg/securityRules/Allow-SSH]
azurerm_network_interface.nic: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/prov
rosoft.Network/networkInterfaces/devops-ca-nic]
azurerm_network_interface_security_group_association.nsg_assoc: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1
Groups/devops-ca-rg/providers/Microsoft.Network/networkInterfaces/devops-ca-nic|/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourc
evops-ca-rg/providers/Microsoft.Network/networkSecurityGroups/devops-ca-nsg]
azurerm_linux_virtual_machine.vm: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/p
Microsoft.Compute/virtualMachines/devops-ca-vm]

No changes. Your infrastructure matches the configuration.
```

Fig 1.2 Terraform plan Command

```
aditya@LAPTOP-J5IGE3UJ:~/devops-ca-project/terraform$ terraform apply
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg]
azurerm_public_ip.public_ip: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers/Micro
soft.Network/publicIPAddresses/devops-ca-public-ip]
azurerm_virtual_network.vnet: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers/Micr
osoft.Network/virtualNetworks/devops-ca-vnet]
azurerm_network_security_group.nsg: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/provider
s/Microsoft.Network/networkSecurityGroups/devops-ca-nsg]
azurerm_subnet.subnet: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers/Microsoft.N
etwork/virtualNetworks/devops-ca-vnet/subnets/devops-ca-subnet]
azurerm_network_security_rule.ssh: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers
/Microsoft.Network/networkSecurityGroups/devops-ca-nsg/securityRules/Allow-SSH]
azurerm_network_security_rule.http: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/provider
s/Microsoft.Network/networkSecurityGroups/devops-ca-nsg/securityRules/Allow-HTTP]
azurerm_network_interface.nic: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers/Mic
rosoft.Network/networkInterfaces/devops-ca-nic]
```

Fig 1.3 Terraform Apply Command

```
azurerm_network_interface.nic: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers/Mic
rosoft.Network/networkInterfaces/devops-ca-nic]
azurerm_network_interface_security_group_association.nsg_assoc: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resource
Groups/devops-ca-rg/providers/Microsoft.Network/networkInterfaces/devops-ca-nic|/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/d
evops-ca-rg/providers/Microsoft.Network/networkSecurityGroups/devops-ca-nsg]
azurerm_linux_virtual_machine.vm: Refreshing state... [id=/subscriptions/0164e810-6295-42eb-b55b-f79d61f4ddd1/resourceGroups/devops-ca-rg/providers/
Microsoft.Compute/virtualMachines/devops-ca-vm]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

public_ip_address = "4.223.136.116"
```

Fig 1.4 Terraform Apply Command and Public IP obtained

6. Configuration Management Using Ansible

After setting up the infrastructure, Ansible was used to set up the VM automatically. Ansible is configuration management tool that doesn't need an agent and communicates over ssh. It doesn't need any extra software on the nodes it manages.

The playbook did the following things:

Updating system packages

Installing Docker

Installing Git

Enabling and starting Docker service

Preparing the server for application deployment

Execution command:

Ansible-playbook -I inventory.ini deploy.yml

Running Ansible playbooks more than once does not cause the system to malfunction because they are idempotent. In order to maintain consistent configuration, task that have already been finished are skipped.

```
aditya@LAPTOP-J5IGE3UJ:~/devops-ca-project/ansible$ ansible all -i inventory.ini -m ping
4.223.136.116 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

Fig 2.1 Ansible Ping Command

```
aditya@LAPTOP-J5IGE3UJ:~/devops-ca-project/ansible$ ansible-playbook -i inventory.ini deploy.yml

PLAY [Configure VM for Docker deployment] ***************************************************************************

TASK [Gathering Facts] **********************************************************************************************
ok: [4.223.136.116]

TASK [Update package list] ******************************************************************************************
changed: [4.223.136.116]

TASK [Install Docker and Git] ***************************************************************************************
ok: [4.223.136.116]

TASK [Start Docker service] *****************************************************************************************
ok: [4.223.136.116]

TASK [Add azureuser to docker group] ********************************************************************************
ok: [4.223.136.116]

PLAY RECAP **********************************************************************************************************
4.223.136.116              : ok=5    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Fig 2.2 Ansible Playbook Command

7. Containerization Using Docker

Docker was used to package the application into a container, ensuring portability and isolation.

A Docker file was created using an Nginx base image to serve a static website.

Example:

FROM Nginx:latest

COPY index.html/usr/share/nginx/index.html

EXPOSE 80

Commands used:

docker build -t portfolio .

```
azureuser@devops-ca-vm:~/devops-ca-project/app$ docker build -t portfolio .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.584kB
Step 1/4 : FROM nginx:latest
 ---> fd204fe2f750
Step 2/4 : COPY index.html /usr/share/nginx/html/index.html
 ---> Using cache
 ---> 4f8f70707289
Step 3/4 : EXPOSE 80
 ---> Using cache
 ---> 6263d4d3b61b
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
 ---> Using cache
 ---> 10659f181870
Successfully built 10659f181870
Successfully tagged portfolio:latest
```

Fig 3.1 Docker Build Command

docker run -d -p 80:80 –name portfolio portfolio

```
azureuser@devops-ca-vm:~/devops-ca-project/app$ docker run -d -p 80:80 --name portfolio portfolio
051a5dcd203909dbfa82c156cbad7a4d33a5ef14f1d06aad2d2db594d7561688
azureuser@devops-ca-vm:~/devops-ca-project/app$ docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED          STATUS          PORTS                                   NAMES
051a5dcd2039   portfolio   "/docker-entrypoint.…"   15 seconds ago   Up 14 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp     portfolio
```

Fig 3.2 Docker Run Command

docker ps

```
azureuser@devops-ca-vm:~/devops-ca-project/app$ docker ps
CONTAINER ID   IMAGE       COMMAND                  CREATED          STATUS          PORTS                                   NAMES
051a5dcd2039   portfolio   "/docker-entrypoint.…"   15 seconds ago   Up 14 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp     portfolio
```

Fig 3.3 Docker Ps Command

Port mapping exposed the container to the internet via VM's public IP, making the website accessible externally.

Containerization ensures consistent behaviour across environment and simplifies deployment

8. Continuous Integration and Continuous Deployment (CI/CD)

CI/CD was implemented using GitHub Action

Every time modifications are added to the main branch:

The workflow connects to the VM via SSH

Pulls the latest code

Stops the existing container

Rebuilds the Docker image

Starts a new container

Sensitive credentials such as VM IP, VM USER, and SSH private key were stored securely using GitHub Secrets.

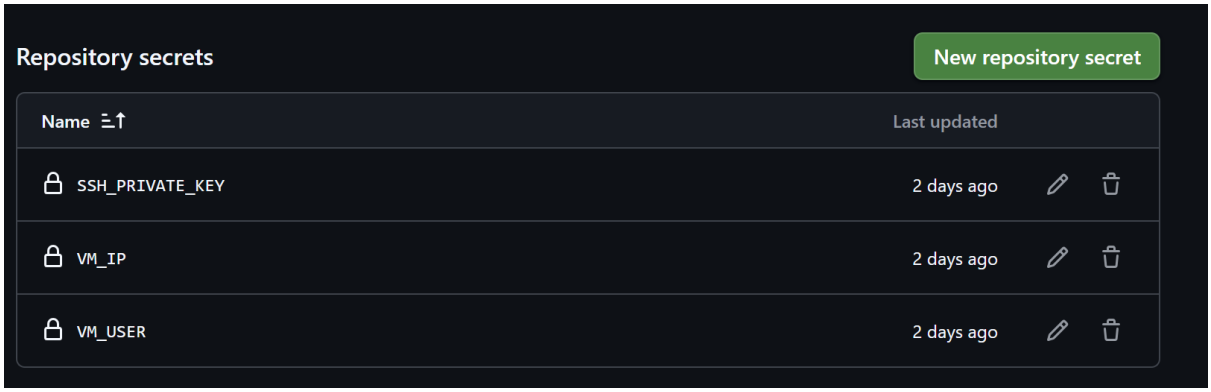

Fig 4.1 GitHub Repository Secrets

This automation eliminates manual deployment and ensures rapid updates.

9. Security and Cost Management

Several security controls were implemented:

SSH key-based authentication

Limited NSG rules

Secure handling of credentials

Minimal open ports

Cost management was addressed by using low-cost VM sizes and destroying resources after testing to avoid unnecessary charges

Responsible cloud usage is essential in real-world deployments

---

10. Reflection and Learning Outcomes

This project provided hands-on experience with modern DevOps tools and cloud technologies. Key learning include

Designing cloud infrastructure using IaC

Understanding Azure networking concepts

Automating server configuration

Containerizing applications

Implementing CI/CD pipelines

Troubleshooting deployment issues

Working within a Linux environment improved familiarity with industry-standard workflows.

---

11. Conclusion

This project successfully implemented an automated container deployment pipeline using terraform, Ansible, Docker, and GitHub Actions on Microsoft Azure.

By combining Infrastructure as code, configuration management, containerization, and CI/CD automation, the system achieves a production-style deployment workflow that mirrors real-world DevOps environments.

The approach improves reliability, reduces configuration errors and enables rapid application updates. This project demonstrates the effectiveness of automation tools in managing cloud-based infrastructure and application deployment provides guidance on implementing CI/CD workflows, automation pipelines, and secure deployment processes using GitHub Actions. Using GitHub Actions, deployments are triggered automatically whenever changes are pushed to the main branch, eliminating the need for manual server access.

12. References

Hashi Corp(2024) Terraform Documentation. Hashi Corp Inc. Available at: http://www.terraform.io/docs

This documentation provides a comprehensive guidance o using terraform for infrastructure as code. Including resource provisioning, state management, and deployment automation.

Red Hat (2024) Ansible Documentation. Red Hat, Inc Available at: http://docs.ansible.com

This source explains Ansible's agentless architecture, playbook syntax, and automation capabilities used for configuration management and system provisioning.

Docker Inc. (2024) Azure Virtual Machines Documentation. Microsoft Learn. Available at: http://learn.microsoft.com/azure

This documentation outlines deployment, configuration, networking and security management of virtual machines in the Microsoft Azure cloud environment.

GitHub (2024) GitHub Actions Documentation. GitHub, Inc. Available at: https://docs.github.com/actions

---

13. Annexure

13.1 Annexure A – Terraform Configuration Snippet (main.tf)

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

```
# ----------------------------
# Resource Group
# ----------------------------
resource "azurerm_resource_group" "rg" {
  name     = "devops-ca-rg"
  location = "Sweden Central"
}

# ----------------------------
# Virtual Network
# ----------------------------
resource "azurerm_virtual_network" "vnet" {
  name                = "devops-ca-vnet"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

# ----------------------------
# Subnet (separate resource)
# ----------------------------
resource "azurerm_subnet" "subnet" {
  name                 = "devops-ca-subnet"
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}

# ----------------------------
# Network Security Group
# ----------------------------
resource "azurerm_network_security_group" "nsg" {
  name                = "devops-ca-nsg"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

# Allow SSH
resource "azurerm_network_security_rule" "ssh" {
  name                        = "Allow-SSH"
```

```
  priority                   = 1001
  direction                  = "Inbound"
  access                     = "Allow"
  protocol                   = "Tcp"
  source_port_range          = "*"
  destination_port_range     = "22"
  source_address_prefix      = "*"
  destination_address_prefix  = "*"
  resource_group_name        = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}

# Allow HTTP
resource "azurerm_network_security_rule" "http" {
  name                       = "Allow-HTTP"
  priority                   = 1002
  direction                  = "Inbound"
  access                     = "Allow"
  protocol                   = "Tcp"
  source_port_range          = "*"
  destination_port_range     = "80"
  source_address_prefix      = "*"
  destination_address_prefix  = "*"
  resource_group_name        = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}

# ----------------------------
# Public IP
# ----------------------------
resource "azurerm_public_ip" "public_ip" {
  name                = "devops-ca-public-ip"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Static"
  sku                 = "Standard"
}

# ----------------------------
# Network Interface
# ----------------------------
```

```
resource "azurerm_network_interface" "nic" {
  name              = "devops-ca-nic"
  location          = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.public_ip.id
  }
}

# Associate NSG to NIC
resource "azurerm_network_interface_security_group_association" "nsg_assoc" {
  network_interface_id      = azurerm_network_interface.nic.id
  network_security_group_id = azurerm_network_security_group.nsg.id
}

# ----------------------------
# Linux VM (Gen2 compatible)
# ----------------------------
resource "azurerm_linux_virtual_machine" "vm" {
  name              = "devops-ca-vm"
  resource_group_name = azurerm_resource_group.rg.name
  location          = azurerm_resource_group.rg.location

  size = "Standard_D2as_v5"

  admin_username = "azureuser"

  network_interface_ids = [
    azurerm_network_interface.nic.id,
  ]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("~/.ssh/id_rsa.pub")
  }

  disable_password_authentication = true
```

```
  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts-gen2"
    version   = "latest"
  }
}

# ----------------------------
# Output Public IP
# ----------------------------
output "public_ip_address" {
  value = azurerm_public_ip.public_ip.ip_address
}
```

13.2 Annexure B – Ansible Playbook Snippet (deploy.yml)

```
name: Configure VM for Docker deployment
  hosts: web
  become: yes

  tasks:

    - name: Update package list
      apt:
        update_cache: yes

    - name: Install Docker and Git
      apt:
        name:
          - docker.io
          - git
        state: present

    - name: Start Docker service
```

```
  service:
    name: docker
    state: started
    enabled: yes

- name: Add azureuser to docker group
  user:
    name: azureuser
```

The above playbook automates server configuration by installing required dependencies, cloning the application repository, building the Docker image, and deploying the container. The playbook ensures repeatable and consistent server setup. Additionally, it reduces manual intervention and minimizes configuration errors that often occur during traditional deployments. By defining tasks in a structured and declarative manner, Ansible enables reliable provisioning across multiple environments. This approach also improves scalability, as the same playbook can be reused to configure additional servers, ensuring uniform system configuration and faster deployment cycles.

13.3 Annexure C – Project Architecture

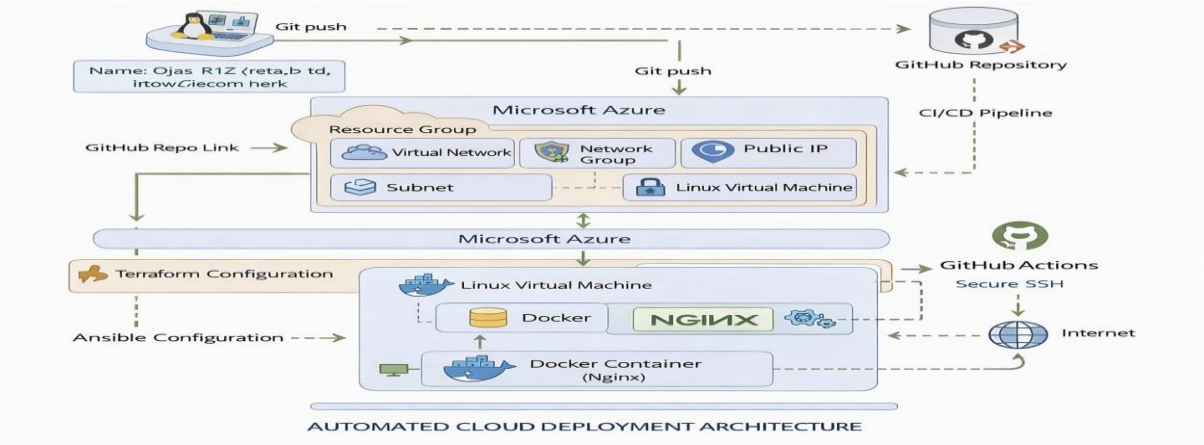

Fig 13.3 Project Architecture Diagram

THANK YOU